实验一 组合逻辑设计

一、实验目的

1 掌握 Verilog 语言和 Vivado、Logisim 开发平台的使用；

2 掌握基础组合逻辑电路的设计和测试方法。

二、实验内容（用 Logisim 或 Vivado 实现）

1 基础门电路（多输入门电路、复用器等）的设计和测试；

2 基础功能模块（编码器、译码器等）的设计与测试。

三、实验要求

1 掌握 Vivado 或 Logisim 开发工具的使用，掌握以上电路的设计和测试方法；

2 记录设计和调试过程（Verilog 代码/电路图/表达式/真值表，Vivado 仿真结果，Logisim 验证结果等）；

3 分析 Vivado 仿真波形/Logism 验证结果，注重输入输出之间的对应关系。

四、实验过程及分析

（一）多输入门电路

1、代码：

```
module mulinput(
    input a, b, c, d, e,
    output x
);
    assign x = ~(a & ~b & c & (d | e));
endmodule
```

逻辑功能：实现多输入逻辑函数 x = ~(a & ~b & c & (d | e))

电路特点：使用连续赋值语句，结构清晰

真值特征：当 a=1, b=0, c=1, 且(d=1 或 e=1)时，x=0

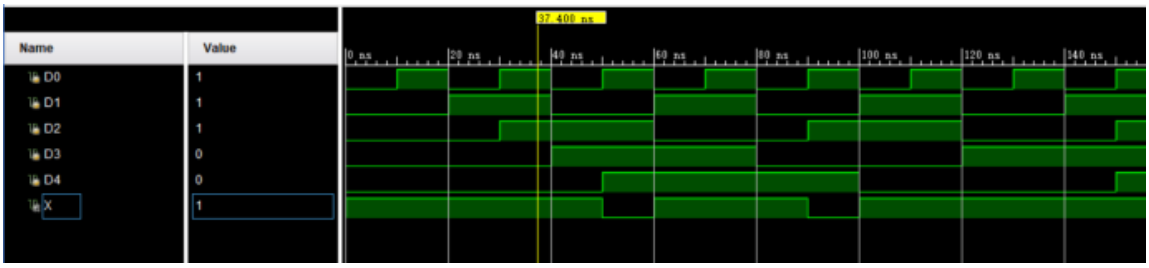
2、测试文件：

```
module sim_mulinput;
    reg D0, D1, D2, D3, D4;
    wire X;
    mulinput in(D0,D1,D2,D3,D4,X);
    initial begin
        D0 = 0; D1 = 0; D2 = 0; D3 = 0; D4 = 0;
        fork
            repeat(100) #10 D0 = ~D0;
            repeat(50) #20 D1 = ~D1;
```

```
            repeat(33) #30 D2 = ~D2;
            repeat(25) #40 D3 = ~D3;
            repeat(20) #50 D4 = ~D4;
        join
    end
endmodule
```

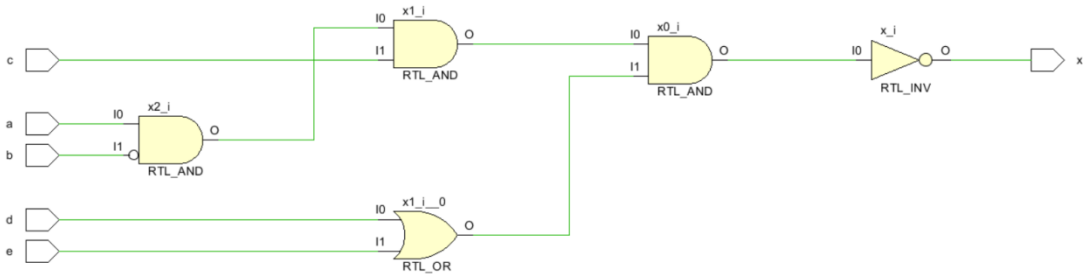
测试策略：采用 fork-join 并行执行，使用不同频率驱动各输入信号，确保充分的测试覆盖率。

3、SIMULATION：



分析：当 a=1，b=0，c=1，且(d=1 或 e=1)时，x=0，其他情况下 x=1，符合预期。

4、RTL ANALYSIS:



（二）二路复用器：

1、代码：

```
module mux21(
    input D0, input D1,
    input S,
    output Y
);
    reg temp;
    always@(*)
    begin
        case(S)
            1'b0 : temp = D0;
```

```
            1'b1 : temp = D1;
            default: temp = D0;
        endcase
    end
    assign Y = temp;
endmodule
```

功能：2 选 1 多路复用器，S=0 时 Y=D0，S=1 时 Y=D1

实现：使用 always 块和 case 语句

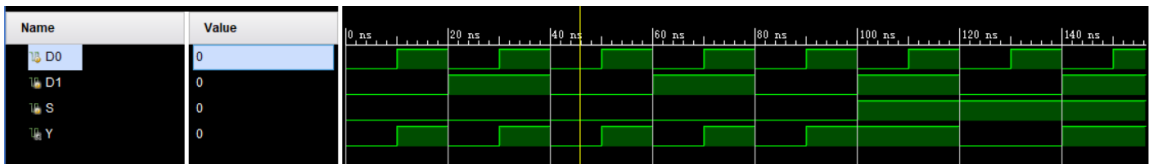
特点：代码结构清晰，包含 default 分支提高健壮性

2、仿真文件：

```
module sim_mux21;
    reg D0, D1;
    reg S;
    wire Y;
    mux21 mux(D0, D1, S, Y);
    initial
    begin
        D0 = 0; D1 = 0; S = 0;
        fork
            repeat(20) #10 D0 = ~D0;
            repeat(10) #20 D1 = ~D1;
            repeat(2) #100 S = ~S;
        join
    end
endmodule
```

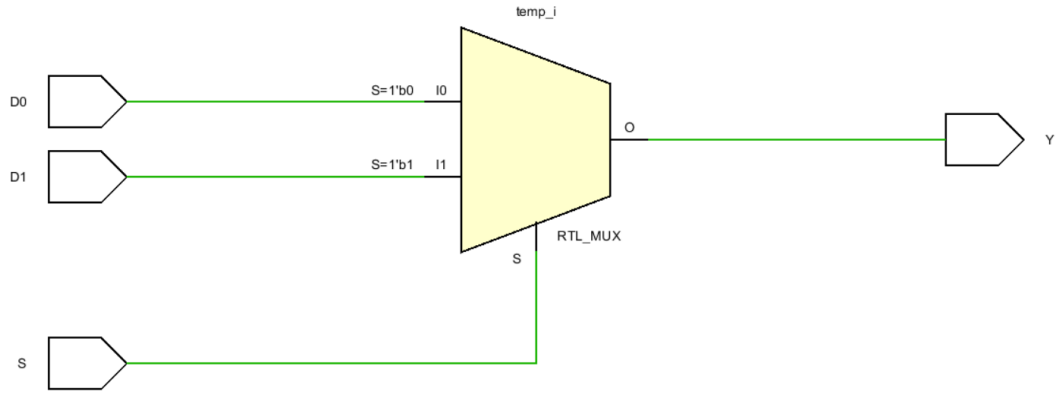
测试策略：采用 fork-join 并行执行，使用不同频率驱动各输入信号，确保充分的测试覆盖率。

3、SIMULATION:



分析：S=0 时 Y=D0，S=1 时 Y=D1，符合预期。

4、RTL ANALYSIS:



（三）四路复用器：

1、代码：

```
module mux41(
    input D0, input D1, input D2, input D3,
    input [1:0] S,
    output Y
);
    wire out0, out1;

    // 使用两个二路复用器处理低两位选择
    mux21 mux0(.D0(D0), .D1(D1), .S(S[0]), .Y(out0));
    mux21 mux1(.D0(D2), .D1(D3), .S(S[0]), .Y(out1));

    // 使用第三个二路复用器处理高位选择
    mux21 mux2(.D0(out0), .D1(out1), .S(S[1]), .Y(Y));
endmodule
```

层次化设计：采用模块化设计方法，使用三个二路复用器构建四路复用器

结构：第一级处理低位选择，第二级处理高位选择

优点：代码复用性高，结构清晰，便于维护扩展

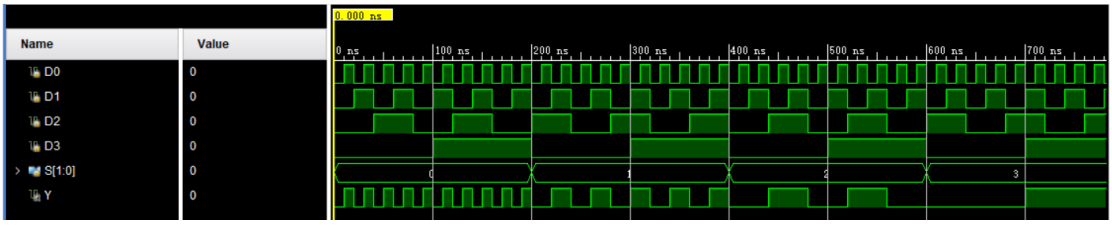
2、仿真文件：

```
module sim_mux41;
    reg D0, D1, D2, D3;
    reg [1:0] S;
    wire Y;
    mux41 mux(D0, D1, D2, D3, S, Y);
    initial
    begin
        D0 = 0; D1 = 0; D2 = 0; D3 = 0; S = 2'b00;
        fork
            repeat(100) #10 D0 = ~D0;
```

```
            repeat(50) #20 D1 = ~D1;
            repeat(25) #40 D2 = ~D2;
            repeat(10) #100 D3 = ~D3;
            repeat(5) #200 S = S + 1;
        join
    end
endmodule
```

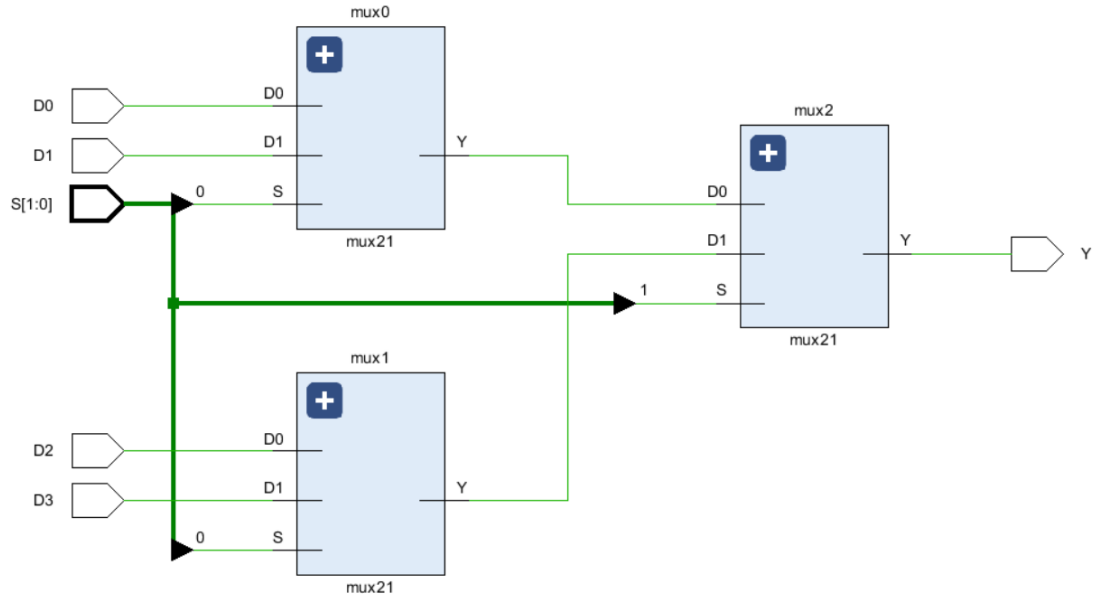
测试策略：采用 fork-join 并行执行，使用不同频率驱动各输入信号，确保充分的测试覆盖率。

3、SIMULATION：



分析：S=0 时，Y=D0；S=1 时，Y=D1；S=2 时，Y=D2；S=3 时，Y=D3。符合功能预期。

4、RTL ANALYSIS:



（四）三态缓存器实现复用器

1、代码：

```
module mux41_tri(
    input D0, input D1, input D2, input D3,
    input [1:0] S,
    output Y
);
    assign Y = (~S[1] & ~S[0]) ? D0 : 1'bz;
```

```
    assign Y = (~S[1] & S[0]) ? D1 : 1'bz;
    assign Y = (S[1] & ~S[0]) ? D2 : 1'bz;
    assign Y = (S[1] & S[0]) ? D3 : 1'bz;
endmodule
```

实现原理：利用三态门的高阻态特性实现多路选择，每个输入对应一个三态缓存器，通过选择信号控制哪个三态门处于有效状态

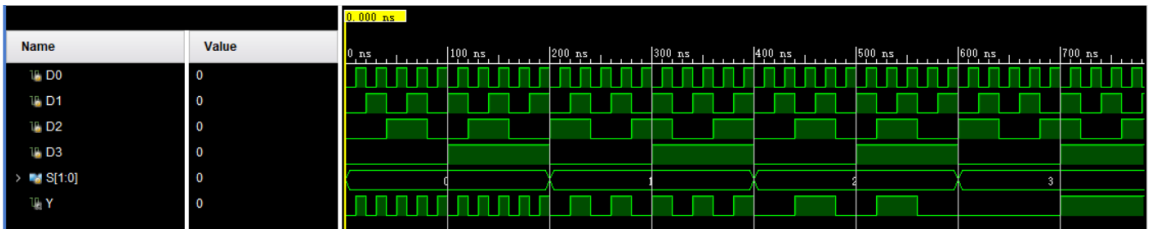
Verilog 实现：使用条件运算符实现三态逻辑，当选择条件满足时，输出对应的数据输入，当选择条件不满足时，输出高阻态(z)

2、仿真文件：

```
module sim_mux41_tri;
    reg D0, D1, D2, D3;
    reg [1:0] S;
    wire Y;
    mux41_tri mux(D0, D1, D2, D3, S, Y);
    initial
    begin
        D0 = 0; D1 = 0; D2 = 0; D3 = 0; S = 2'b00;
        fork
            repeat(100) #10 D0 = ~D0;
            repeat(50) #20 D1 = ~D1;
            repeat(25) #40 D2 = ~D2;
            repeat(10) #100 D3 = ~D3;
            repeat(5) #200 S = S + 1;
        join
    end
endmodule
```

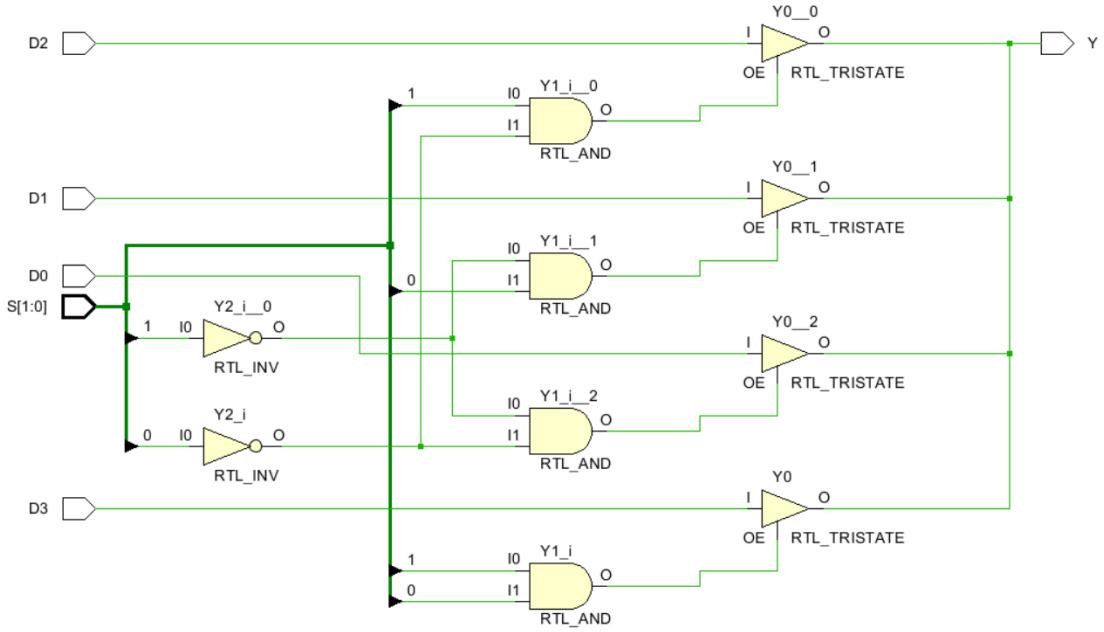
测试策略：同上（四路复用器）。

3、SIMULATION：



分析：同上（四路复用器）。

4、RTL ANALYSIS:



（五）编码器：

1、代码：

（1）逻辑表达式实现编码器：

```
module encoder83_assign(
    input [7:0] I,
    output [2:0] Y
);
    assign Y[0] = I[1] | I[3] | I[5] | I[7];
    assign Y[1] = I[2] | I[3] | I[6] | I[7];
    assign Y[2] = I[4] | I[5] | I[6] | I[7];
endmodule
```

功能：实现 8 线-3 线编码器功能，输入 8 位信号，输出 3 位二进制编码，使用逻辑或运算实现编码功能

（2）case 实现编码器：

```
module encoder83_case(I, Y);
    input I;
    output Y;
    wire[7:0] I;
    reg[3:1] Y;

    always @(I) begin
        case(I)
            8'b0000_0001 : Y = 3'b000;
            8'b0000_0010 : Y = 3'b001;
            8'b0000_0100 : Y = 3'b010;
```

```
            8'b0000_1000 : Y = 3'b011;
            8'b0001_0000 : Y = 3'b100;
            8'b0010_0000 : Y = 3'b101;
            8'b0100_0000 : Y = 3'b110;
            8'b1000_0000 : Y = 3'b111;
        endcase
    end
endmodule
```

分析：采用 case 语句描述编码逻辑，每个输入位对应一个 case 分支，代码可读性好，易于理解。

（3）case 实现优先编码器：

```
module pre_encoder83_case(I, Y);
    input I;
    output Y;
    wire[7:0] I;
    reg[3:1] Y;

    always @(I) begin
        casex(I)
            8'b0000_0001 : Y = 3'b000;
            8'b0000_001X : Y = 3'b001;
            8'b0000_01XX : Y = 3'b010;
            8'b0000_1XXX : Y = 3'b011;
            8'b0001_XXXX : Y = 3'b100;
            8'b001X_XXXX : Y = 3'b101;
            8'b01XX_XXXX : Y = 3'b110;
            8'b1XXX_XXXX : Y = 3'b111;
            default: Y = 3'b000;
        endcase
    end
endmodule
```

分析：使用 casex 语句处理不确定位，实现了优先级编码功能，高位的输入具有更高的优先级。

（4）if-else 实现优先编码器：

```
module pre_encoder83_if(I, Y);
    input [7:0] I;
    output [2:0] Y;
    reg [2:0] Y;
    always @(*) begin
        if(I[7] == 1) Y = 3'b111;
        else if(I[6] == 1) Y = 3'b110;
```

```
        else if(I[5] == 1) Y = 3'b101;
        else if(I[4] == 1) Y = 3'b100;
        else if(I[3] == 1) Y = 3'b011;
        else if(I[2] == 1) Y = 3'b010;
        else if(I[1] == 1) Y = 3'b001;
        else if(I[0] == 1) Y = 3'b000;
        else Y = 3'b000;
    end
endmodule
```

分析：采用嵌套 if-else 结构，从最高位到最低位依次判断，实现了优先级编码逻辑。

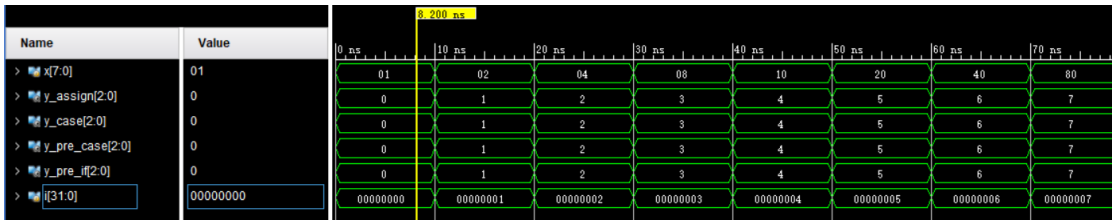
2、仿真文件：

```
module sim_encoder83();
    reg [7:0] x;
    wire [2:0] y_assign, y_case, y_pre_case, y_pre_if;
    integer i;

    initial begin
        x = 1;
        for(i = 0; i < 7; i = i + 1) #10 x = x * 2;
        #10 x = 128;
        while(x > 0) #5 x = x - 1;
    end
    encoder83_assign encoder83_assign_1(.I(x), .Y(y_assign));
    encoder83_case encoder83_case_1(.I(x), .Y(y_case));
    pre_encoder83_case pre_encoder83_case_1(.I(x), .Y(y_pre_case));
    pre_encoder83_if pre_encoder83_if_1(.I(x), .Y(y_pre_if));
endmodule
```

测试策略：将四种方式下完成的编码器作为四个输出，并进行实例化。采用 for 循环改变 x 值，观察 y 的对应变化。

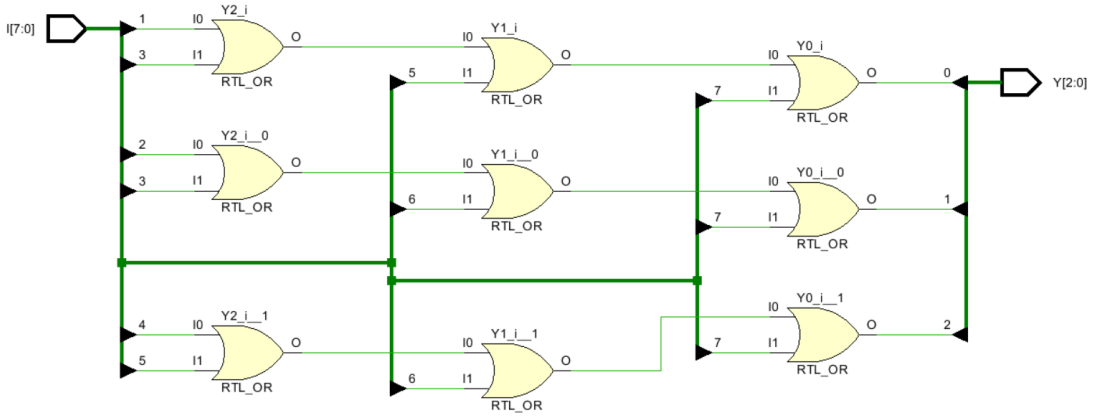
3、SIMULATION：



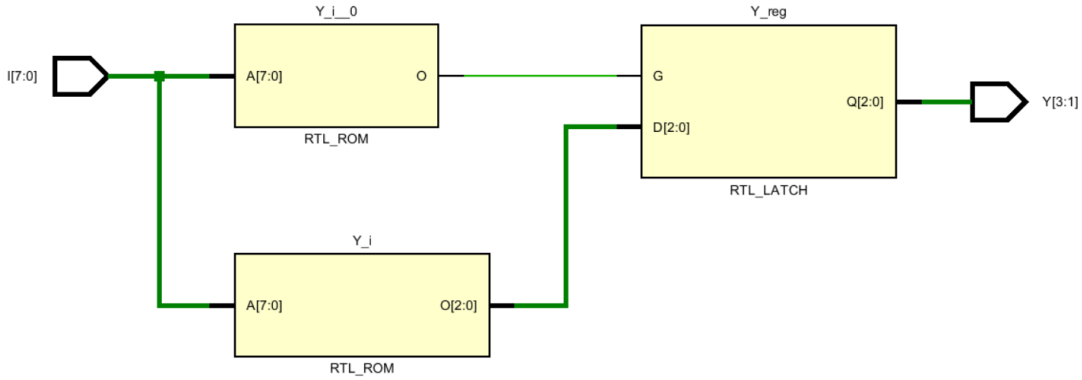
分析：四种方式都正确地实现了编码器的预期功能。

4、RTL ANALYSIS:

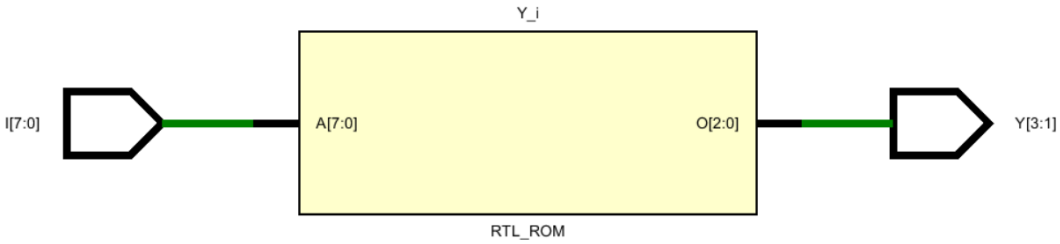
（1）encoder83_assign：



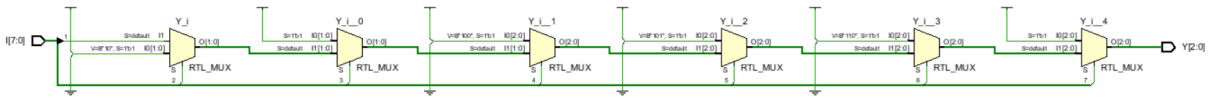
（2）encoder83_case:



（3）pre_encoder83_case:



（4）pre_encoder83_if:



（六）译码器：

1、代码：

```
module Decoder38(A, Y);
    input [2:0] A;
    output reg [7:0] Y;
    integer i;
```

```
    always @(A) begin
        Y = 8'b1111_1111;
        for(i = 0; i<=7; i = i + 1)
            if (A == i)
                Y[i] = 0;
            else
                Y[i] = 1;
    end
endmodule
```

功能描述：实现 3 线-8 线译码器功能，输入 3 位二进制信号，输出 8 位译码结果，采用低电平有效输出。

实现特点：使用 for 循环遍历所有可能的输入值，当输入等于某个值时，对应的输出位清零，其他输出位保持为 1。

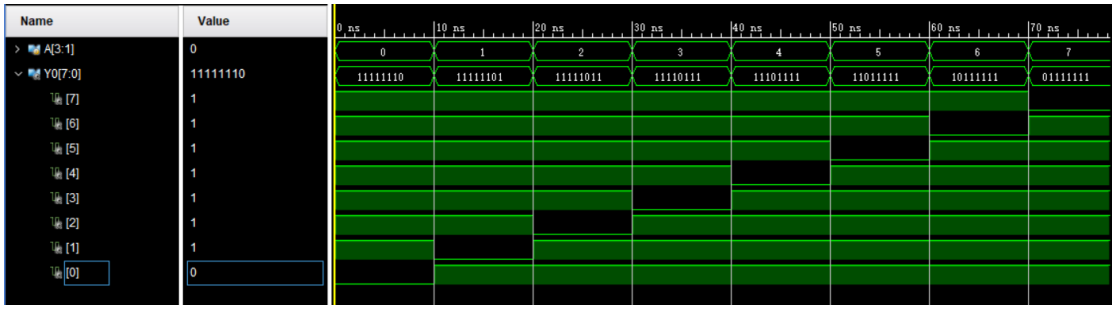
设计优势：代码简洁，避免了重复的 if-else 结构。

2、仿真文件：

```
module sim_Decoder38;
    reg [3:1] A;
    wire [7:0] Y0;
    decoder38 test(.A(A), .Y(Y0));
    initial begin
        A = 3'b000;
        repeat (10) #10 A = A + 1;
    end
endmodule
```

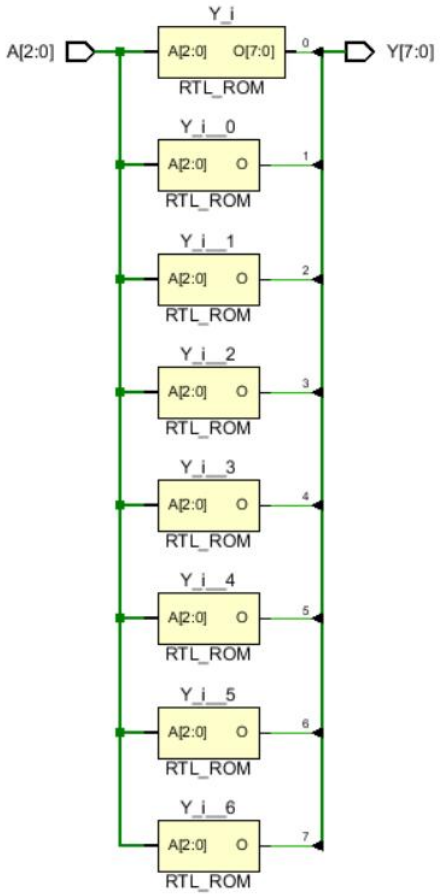
测试策略：不断增加输入 A 的值，观察输出 Y 是否按预期进行译码。

3、SIMULATION：



分析：Y 对应输入 A 的位为 0，其余位为 1，符合译码器的预期功能。

4、RTL ANALYSIS:



五、调试和心得体会

1、实验结果分析

通过 Vivado 仿真验证，所有设计的组合逻辑电路均能正确实现预期功能：

①多输入门电路:成功实现了复杂的逻辑函数,验证了 Verilog 逻辑运算符的正确使用。

②复用器：从二路复用器到四路复用器，层次化设计方法展现了良好的可扩展性。

③编码器：多种实现方式（逻辑表达式、case 语句、if-else）的比较验证了不同描述方式的优劣。

④译码器：使用循环结构简化了代码，提高了可读性和可维护性。

2、技术收获

①Verilog 编程能力：回忆 Verilog 编程的方式与 Vivado 平台的使用，为以后的实验打下编程基础。

②测试方法：回忆了使用 fork-join 结构创建复杂的测试信号，通过设计并行信号对系统进行了全面的仿真以确定正确性。

③层次化设计：回忆了如何通过模块实例化构建更复杂的系统。

3、遇到的问题及解决方案

①语法错误：初期对 Verilog 语法和 Vivado 平台的使用产生遗忘，通过查阅资料和反复调试解决了问题。

②逻辑错误：在设计复杂逻辑时出现过功能错误，通过真值表验证和逐步调试找到问题

根源。如 casex 与 case 语句的使用场景不同，在开始时我并没有注意到这个问题，反复调试后发现问题根源并加深了对 casex 语句的认识。

③仿真设置：测试文件的编写需要仔细考虑时序关系，通过多次尝试优化了测试策略，保证仿真结果直观且全面。

4、心得体会

本次实验主要通过基础的组合逻辑电路练习 Vivado 平台与 Verilog 语言的使用。通过这次实验，我掌握了 Verilog 基础语法、数字电路的设计思路与仿真文件的设计思路，重温了 Vivado 开发的流程，掌握了 begin、initial、case、fork-join 等语句的用法，并了解了 casex 以及 x、z 状态的使用场景。这次实验的练习为后续更加复杂的系统设计打下了坚实基础。